

Go net/http Package

The Complete Reference

HTTP Servers · HTTP Clients · Transport · ServeMux · Timeouts · Patterns

Based on: Mastering Go (4th Ed.) • pkg.go.dev/net/http

1. Package Overview

The net/http package is part of Go's standard library and provides everything needed to build production-grade HTTP servers and clients. It handles TCP connection management, request parsing, response writing, keep-alive connections, TLS, HTTP/2, timeouts, and concurrent request handling — all out of the box.

The package is organized around a small set of core types and a set of functions that work with those types. Understanding the three foundational types — `http.Response`, `http.Request`, and `http.Transport` — is essential before working with the higher-level server and client APIs.

1.1 Key Functions at a Glance

Function / Type	Role
<code>http.ListenAndServe()</code>	Start an HTTP server on a given address with a handler.
<code>http.HandleFunc()</code>	Register a handler function for a URL pattern with the default <code>ServeMux</code> .
<code>http.Get()</code>	Send a simple GET request and return the response.
<code>http.NewRequest()</code>	Build an HTTP request with full control over method, headers, and body.
<code>http.NewRequestWithContext()</code>	Like <code>http.NewRequest()</code> but attaches a context for cancellation and timeouts.
<code>http.Client{}</code>	High-level HTTP client with configurable transport, redirect policy, and timeout.
<code>http.Transport{}</code>	Low-level transport: controls connection pooling, TLS, proxies, and dial behavior.
<code>http.ServeMux{}</code>	HTTP request multiplexer (router) that maps URL patterns to handler functions.
<code>http.NewServeMux()</code>	Create a new, independent <code>ServeMux</code> (as opposed to the global <code>DefaultServeMux</code>).
<code>http.Server{}</code>	Full HTTP server configuration struct: address, handler, timeouts, TLS.
<code>http.HandlerFunc</code>	Adapter type that lets ordinary functions satisfy the <code>http.Handler</code> interface.
<code>http.ResponseWriter</code>	Interface used by handlers to write status codes, headers, and body to the client.

2. Core Data Types

2.1 http.Response

`http.Response` represents the response returned by an HTTP request. Both `http.Client` and `http.Transport` return `*http.Response` values once the response headers have been received. The body is a streaming reader and must be closed after use.

```
type Response struct {
    Status      string      // e.g. "200 OK"
    StatusCode  int         // e.g. 200
    Proto       string      // e.g. "HTTP/1.1" or "HTTP/2.0"
    ProtoMajor  int         // e.g. 1
    ProtoMinor  int         // e.g. 1
    Header      Header      // response headers (map[string][]string)
    Body        io.ReadCloser // response body — MUST be closed
    ContentLength int64       // -1 if unknown
    TransferEncoding []string
    Close        bool        // server wants connection closed after response
    Uncompressed bool        // true if body was decompressed by transport
    Trailer      Header
    Request      *Request    // the request that produced this response
    TLS          *tls.ConnectionState
}
```

The most commonly used fields in practice are `Status`, `StatusCode`, `Header`, `Body`, and `ContentLength`. After reading the body, you must close it to free the underlying TCP connection for reuse:

```
resp, err := http.Get("https://example.com")
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close() // always close the body

fmt.Println(resp.Status) // "200 OK"
fmt.Println(resp.StatusCode) // 200

body, err := io.ReadAll(resp.Body)
if err != nil {
    log.Fatal(err)
}
fmt.Println(string(body))
```

Critical: Always close `resp.Body` — even if you don't read it. Failing to close the body prevents the HTTP connection from being reused and eventually exhausts available connections. Use `defer resp.Body.Close()` immediately after checking the error.

2.2 http.Request

`http.Request` represents an HTTP request as constructed by a client to be sent, or as received by a server from a client. When acting as a server handler, you receive a `*http.Request` argument that contains all information about the incoming request.

```
type Request struct {
    Method string          // "GET", "POST", "PUT", "DELETE", etc.
    URL     *url.URL               // parsed request URL
    Proto   string          // "HTTP/1.1"
    ProtoMajor int
    ProtoMinor int
    Header  Header          // request headers
    Body    io.ReadCloser        // request body (nil for GET)
    GetBody func() (io.ReadCloser, error) // optional body copy fn
    ContentLength int64
    TransferEncoding []string
    Close bool
    Host string            // host:port from the URL or Host header
    Form url.Values          // parsed form data (call ParseForm first)
    PostForm url.Values    // POST/PUT form data only
    MultipartForm *multipart.Form
    Trailer Header
    RemoteAddr string      // client address "IP:port"
    RequestURI string      // unmodified request-target
    TLS *tls.ConnectionState
    Cancel <-chan struct{} // deprecated: use context
    Response *Response // redirect response that caused this request
}
```

In server handlers, the most commonly accessed fields are `Method`, `URL`, `Header`, `Body`, `RemoteAddr`, and the context via `r.Context()`. On the client side, you build a request using `http.NewRequest()` or `http.NewRequestWithContext()` and then send it via `client.Do()`.

```
// Reading request data in a handler
func myHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Println("Method:", r.Method)      // GET, POST, ...
    fmt.Println("Path:", r.URL.Path)       // /api/users
    fmt.Println("Query:", r.URL.RawQuery)  // name=alice&age=30
    fmt.Println("Client:", r.RemoteAddr)   // 127.0.0.1:54321

    // Read a query parameter
    name := r.URL.Query().Get("name")

    // Read a header
    auth := r.Header.Get("Authorization")

    // Read the body (POST/PUT)
    body, _ := io.ReadAll(r.Body)
    defer r.Body.Close()

    _, _ = name, auth
    fmt.Fprintf(w, "Received: %s", body)
```

```
}
```

2.3 http.Transport

`http.Transport` is the low-level HTTP transport mechanism. It handles connection pooling, TLS configuration, proxy setup, dial behavior, and various timeout settings. Every `http.Client` has a `Transport` field — if `nil`, `http.DefaultTransport` is used.

```
type Transport struct {
    // Proxy: function returning the proxy URL for a given request
    Proxy func(*Request) (*url.URL, error)

    // DialContext: custom dialer with context support
    DialContext func(ctx context.Context, network, addr string) (net.Conn,
error)

    // Dial: legacy custom dialer (deprecated; prefer DialContext)
    Dial func(network, addr string) (net.Conn, error)

    // TLS configuration for HTTPS connections
    TLSClientConfig *tls.Config
    TLSHandshakeTimeout time.Duration

    // Connection pooling
    DisableKeepAlives bool
    MaxIdleConns int // total idle connections across all
hosts
    MaxIdleConnsPerHost int // idle connections per host (default
2)
    MaxConnsPerHost int // 0 = no limit
    IdleConnTimeout time.Duration // how long to keep idle connections

    // Timeouts
    ResponseHeaderTimeout time.Duration // time to wait for server headers
    ExpectContinueTimeout time.Duration

    // Buffer sizes
    WriteBufferSize int
    ReadBufferSize int

    // Force HTTP/2 even for http:// URLs
    ForceAttemptHTTP2 bool

    DisableCompression bool
    MaxResponseHeaderBytes int64
}
```

`http.Transport` is safe for concurrent use and should be reused rather than created per request. Creating a new transport per request is a common mistake that causes connection leaks and prevents keep-alive from working.

```
// Custom transport - shared and reused
transport := &http.Transport{
    MaxIdleConns:    100,
    MaxIdleConnsPerHost: 10,
    IdleConnTimeout:  90 * time.Second,
    TLSHandshakeTimeout: 10 * time.Second,
    DisableCompression: false,
}

client := &http.Client{
    Transport: transport,
    Timeout:   30 * time.Second,
}
```

Best Practice: Create one `http.Client` (and one `http.Transport`) per application and reuse it for all requests. The client is goroutine-safe. Creating a new client per request bypasses connection pooling and is much slower.

3. Building HTTP Servers

3.1 The Simplest Server: `http.ListenAndServe`

The quickest way to start an HTTP server is to register handler functions with `http.HandleFunc()` and call `http.ListenAndServe()`. This uses the global `DefaultServeMux` as the router.

```
package main

import (
    "fmt"
    "net/http"
    "time"
)

// Handler function signature: func(http.ResponseWriter, *http.Request)
func myHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Serving: %s\n", r.URL.Path)
    fmt.Printf("Served: %s\n", r.Host) // logged to server terminal
}

func timeHandler(w http.ResponseWriter, r *http.Request) {
    t := time.Now().Format(time.RFC1123)
    fmt.Fprintf(w, "<h1>The current time is:</h1>")
    fmt.Fprintf(w, "<h2>%s</h2>\n", t)
    fmt.Fprintf(w, "Serving: %s\n", r.URL.Path)
}

func main() {
    http.HandleFunc("/time", timeHandler) // specific path first
    http.HandleFunc("/", myHandler)      // catch-all (default handler)

    // addr format: "host:port" - empty host = listen on all interfaces
    err := http.ListenAndServe(":8001", nil)
    if err != nil {
        fmt.Println(err)
    }
}
```

When `http.ListenAndServe()` is called with `nil` as the second argument, it uses `DefaultServeMux` — the global router that `http.HandleFunc()` registers patterns into.

Path Matching Rule: The `/` pattern matches every URL not matched by any more specific pattern. Registering a handler for `/` makes it the default catch-all. More specific patterns (e.g. `/time`) always take precedence over `/`.

```
# Test with curl:
$ curl localhost:8001/           # served by myHandler
Serving: /
```

```
$ curl localhost:8001/time          # served by timeHandler
<h1>The current time is:</h1><h2>Thu, 31 Aug 2023 22:37:37 EEST</h2>
Serving: /time

$ curl localhost:8001/anything     # also served by myHandler (catch-all)
Serving: /anything
```

3.2 http.ResponseWriter

`http.ResponseWriter` is the interface passed to every handler function. It is used to write the HTTP status code, headers, and body back to the client. It implements `io.Writer`.

Method	Description
<code>Header() http.Header</code>	Returns the header map that will be sent. Must be modified before <code>WriteHeader()</code> or <code>Write()</code> .
<code>WriteHeader(statusCode int)</code>	Sends the HTTP response header with the given status code. Can only be called once.
<code>Write([]byte) (int, error)</code>	Writes data to the response body. Calls <code>WriteHeader(200)</code> implicitly if not already called.

```
func handler(w http.ResponseWriter, r *http.Request) {
    // Set response headers BEFORE WriteHeader or Write
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("X-Custom-Header", "myvalue")

    // Send status code
    w.WriteHeader(http.StatusCreated) // 201

    // Write body
    fmt.Fprintf(w, `{"status": "created"}`)
}

// Common status code constants:
// http.StatusOK           = 200
// http.StatusCreated      = 201
// http.StatusNoContent    = 204
// http.StatusBadRequest  = 400
// http.StatusUnauthorized = 401
// http.StatusForbidden    = 403
// http.StatusNotFound     = 404
// http.StatusNotModified  = 304
// http.StatusInternalServerError = 500
```

Important: Once `WriteHeader()` has been called, you cannot change the status code or add headers. Always set headers before writing anything. Calling `Write()` without `WriteHeader()` sends a 200 OK automatically.

3.3 Custom Server with http.Server

For production servers, you should configure an `http.Server` struct directly instead of relying on `http.ListenAndServe()`. This lets you set timeouts, TLS, custom handlers, and other server-level options.

```
type Server struct {
    Addr      string          // address to listen on, e.g. ":8080"
    Handler   http.Handler    // handler to invoke; nil = DefaultServeMux

    // Timeouts (STRONGLY recommended for production)
    ReadTimeout      time.Duration // max time to read entire request
    including body
    ReadHeaderTimeout time.Duration // max time to read request headers only
    WriteTimeout     time.Duration // max time to write response
    IdleTimeout      time.Duration // max time to wait for next request on
    keep-alive

    MaxHeaderBytes int // max bytes in request header; default 1MB
    TLSConfig      *tls.Config
    ErrorLog       *log.Logger
    BaseContext    func(net.Listener) context.Context
    ConnContext    func(ctx context.Context, c net.Conn) context.Context
}
```

```
func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/", defaultHandler)
    mux.HandleFunc("/api/", apiHandler)

    srv := &http.Server{
        Addr:      ":8080",
        Handler:   mux,
        ReadTimeout: 5 * time.Second,
        ReadHeaderTimeout: 2 * time.Second,
        WriteTimeout: 10 * time.Second,
        IdleTimeout: 120 * time.Second,
        MaxHeaderBytes: 1 << 20, // 1 MB
    }

    log.Printf("Starting server on %s", srv.Addr)
    if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
        log.Fatalf("Server error: %v", err)
    }
}
```

Production Requirement: Never deploy an `http.Server` without timeouts. Without `ReadTimeout` and `WriteTimeout`, a slow or malicious client can hold a connection open indefinitely, eventually exhausting server resources (Slowloris attack).

3.4 ServeMux — The HTTP Router

`http.ServeMux` is Go's built-in HTTP request multiplexer (router). It matches incoming request URLs against a list of registered patterns and calls the corresponding handler.

Default ServeMux vs Custom ServeMux

Approach	When to Use	How
DefaultServeMux (global)	Simple apps, prototyping	<code>http.HandleFunc("/path", fn)</code> then <code>http.ListenAndServe(addr, nil)</code>
Custom <code>http.NewServeMux()</code>	Production, testing, multiple servers, avoiding global state	<code>mux := http.NewServeMux();</code> <code>mux.HandleFunc("/path", fn);</code> <code>srv := &http.Server{Handler: mux}</code>

```
// Using the default ServeMux (global, package-level)
http.HandleFunc("/", homeHandler)
http.HandleFunc("/api/users", usersHandler)
http.ListenAndServe(":8080", nil) // nil = use DefaultServeMux

// Using a custom ServeMux (preferred for production)
mux := http.NewServeMux()
mux.HandleFunc("/", homeHandler)
mux.HandleFunc("/api/users", usersHandler)
http.ListenAndServe(":8080", mux)
```

ServeMux Pattern Matching Rules

- Fixed paths like `/api/users` match exactly that URL.
- Subtree patterns ending with `/` like `/api/` match any URL rooted at that path.
- Longer patterns take precedence over shorter ones.
- The pattern `/` is the catch-all: it matches any URL not matched elsewhere.

```
mux := http.NewServeMux()
mux.HandleFunc("/", catchAll)           // matches everything not matched below
mux.HandleFunc("/about", aboutHandler) // matches /about exactly
mux.HandleFunc("/api/", apiHandler)    // matches /api/, /api/users,
//api/v1/...
mux.HandleFunc("/api/users", usersHandler) // matches /api/users exactly
// (beats /api/)
```

Handle vs HandleFunc

ServeMux provides two registration methods. HandleFunc takes a function directly. Handle takes any type implementing the http.Handler interface (which requires a ServeHTTP method):

```
// HandleFunc - takes a plain function
mux.HandleFunc("/path", func(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "hello")
})

// Handle - takes an http.Handler interface
// http.HandlerFunc converts a plain function to an http.Handler
mux.Handle("/path", http.HandlerFunc(myFunc))

// http.HandlerFunc adapter - makes a function satisfy http.Handler
type HandlerFunc func(ResponseWriter, *Request)
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) { f(w, r) }

// Implementing http.Handler interface manually
type MyHandler struct{ message string }
func (h *MyHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, h.message)
}
mux.Handle("/custom", &MyHandler{message: "hello from struct"})
```

3.5 Extracting URL Parameters

Since Go's built-in ServeMux does not support URL parameters (like /users/:id), parameters are typically passed as part of the URL path and extracted by splitting the path string. Go 1.22 added basic path parameter support with {name} syntax.

```
// Technique used in the statistics web service:
// URL: /delete/datasetName or /insert/name/v1/v2/v3/

func deleteHandler(w http.ResponseWriter, r *http.Request) {
    // r.URL.Path = "/delete/myDataset"
    parts := strings.Split(r.URL.Path, "/")
    // parts = ["", "delete", "myDataset"]

    if len(parts) < 3 || parts[2] == "" {
        w.WriteHeader(http.StatusBadRequest)
        fmt.Fprintln(w, "Missing dataset name")
        return
    }
    dataset := parts[2]
    fmt.Fprintf(w, "Deleting: %s\n", dataset)
}

// Extracting variable-length parameters (e.g. /insert/name/1.0/2/3/4/)
func insertHandler(w http.ResponseWriter, r *http.Request) {
    parts := strings.Split(r.URL.Path, "/")
    // parts = ["", "insert", "name", "1.0", "2", "3", "4", ""]
}
```

```

    if len(parts) < 4 {
        w.WriteHeader(http.StatusBadRequest)
        fmt.Fprintln(w, "Not enough arguments")
        return
    }
    name := parts[2]
    rawValues := parts[3:]
    var values []float64
    for _, v := range rawValues {
        if f, err := strconv.ParseFloat(v, 64); err == nil {
            values = append(values, f)
        }
    }
    fmt.Fprintf(w, "Inserted %s with %d values\n", name, len(values))
}

```

```

// Go 1.22+: native path parameters and method routing
mux := http.NewServeMux()
mux.HandleFunc("GET /users/{id}", func(w http.ResponseWriter, r
    *http.Request) {
    id := r.PathValue("id") // built-in path parameter extraction
    fmt.Fprintf(w, "User ID: %s", id)
})

```

3.6 Complete Web Service Example (Statistics Server)

The following is the complete structure of a multi-endpoint HTTP service using a custom ServeMux and http.Server, based on the statistics application from the book:

```

// handlers.go
package main

import (
    "fmt"
    "log"
    "net/http"
    "strings"
    "strconv"
)

const PORT = ":1234"

func defaultHandler(w http.ResponseWriter, r *http.Request) {
    log.Printf("Serving %s from %s", r.URL.Path, r.Host)
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "Thanks for visiting!")
}

func listHandler(w http.ResponseWriter, r *http.Request) {
    log.Printf("Serving %s from %s", r.URL.Path, r.Host)
    w.WriteHeader(http.StatusOK)
}

```

```

    fmt.Fprintf(w, "%s", list()) // list() is a domain function
}

func statusHandler(w http.ResponseWriter, r *http.Request) {
    log.Printf("Serving %s from %s", r.URL.Path, r.Host)
    w.WriteHeader(http.StatusOK)
    fmt.Fprintf(w, "Total entries: %d\n", len(data))
}

func deleteHandler(w http.ResponseWriter, r *http.Request) {
    parts := strings.Split(r.URL.Path, "/")
    if len(parts) < 3 {
        w.WriteHeader(http.StatusNotFound)
        fmt.Fprintln(w, "Not found:", r.URL.Path)
        return
    }
    log.Printf("Serving %s from %s", r.URL.Path, r.Host)
    dataset := parts[2]
    if err := deleteEntry(dataset); err != nil {
        w.WriteHeader(http.StatusNotFound)
        fmt.Fprintf(w, "%s\n", err.Error())
        return
    }
    w.WriteHeader(http.StatusOK)
    fmt.Fprintf(w, "%s deleted!\n", dataset)
}

func searchHandler(w http.ResponseWriter, r *http.Request) {
    parts := strings.Split(r.URL.Path, "/")
    if len(parts) < 3 {
        w.WriteHeader(http.StatusNotFound)
        fmt.Fprintln(w, "Not found:", r.URL.Path)
        return
    }
    dataset := parts[2]
    t := search(dataset)
    if t == nil {
        w.WriteHeader(http.StatusNotFound)
        fmt.Fprintf(w, "Could not be found: %s\n", dataset)
        return
    }
    w.WriteHeader(http.StatusOK)
    fmt.Fprintf(w, "%s %d %f %f\n", t.Name, t.Len, t.Mean, t.StdDev)
}

func insertHandler(w http.ResponseWriter, r *http.Request) {
    parts := strings.Split(r.URL.Path, "/")
    if len(parts) < 4 {
        w.WriteHeader(http.StatusBadRequest)
        fmt.Fprintln(w, "Not enough arguments:", r.URL.Path)
        return
    }
    dataset := parts[2]
    var values []float64
    for _, v := range parts[3:] {
        if f, err := strconv.ParseFloat(v, 64); err == nil {
            values = append(values, f)
        }
    }
}

```

```

    }
    entry := process(dataset, values)
    if err := insert(&entry); err != nil {
        w.WriteHeader(http.StatusNotModified)
        fmt.Fprintln(w, "Failed to add record")
        return
    }
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "New record added successfully")
}

```

```

// stats.go - main() wiring
func main() {
    mux := http.NewServeMux()

    // Register routes - note both /search and /search/ are registered
    mux.Handle("/list",      http.HandlerFunc(listHandler))
    mux.Handle("/status",    http.HandlerFunc(statusHandler))
    mux.Handle("/insert/",   http.HandlerFunc(insertHandler))
    mux.Handle("/insert",    http.HandlerFunc(insertHandler))
    mux.Handle("/delete/",   http.HandlerFunc(deleteHandler))
    mux.Handle("/search/",   http.HandlerFunc(searchHandler))
    mux.Handle("/search",    http.HandlerFunc(searchHandler))
    mux.Handle("/",          http.HandlerFunc(defaultHandler))

    // mux.Handle can be replaced by mux.HandleFunc:
    // mux.HandleFunc("/list", listHandler) - equivalent

    srv := &http.Server{
        Addr:         PORT,
        Handler:      mux,
        IdleTimeout:  10 * time.Second,
        ReadTimeout:  time.Second,
        WriteTimeout: time.Second,
    }

    fmt.Println("Ready to serve at", PORT)
    if err := srv.ListenAndServe(); err != nil {
        fmt.Println(err)
    }
}

```

Permanent Redirect Behavior: If you register only `/delete/` (with trailing slash) but a client requests `/delete` (without), the `ServeMux` automatically replies with a 301 Moved Permanently redirect to `/delete/`. To accept both, register both patterns explicitly.

4. Building HTTP Clients

4.1 Simple GET: `http.Get()`

`http.Get()` is the simplest way to make an HTTP GET request. It returns an `*http.Response` and an error. It uses `http.DefaultClient` internally, which means no custom timeout or transport configuration.

```
package main

import (
    "fmt"
    "io"
    "net/http"
    "os"
)

func main() {
    resp, err := http.Get("https://www.golang.org")
    if err != nil {
        fmt.Println(err)
        os.Exit(1)
    }
    defer resp.Body.Close() // ALWAYS close the body

    // Stream the response body to stdout
    _, err = io.Copy(os.Stdout, resp.Body)
    if err != nil {
        fmt.Println(err)
    }
}
```

Warning: `http.Get()` (and `http.DefaultClient`) has no timeout set. In production code, always use a custom `http.Client` with a `Timeout` field set. A server that never responds will hang your program forever.

4.2 Custom Client: `http.Client` with Timeout

For production use, always create a custom `http.Client` with an explicit timeout. The `Timeout` field covers the entire request lifecycle (dial, TLS, redirect, read body).

```
client := &http.Client{
    Timeout: 15 * time.Second, // total request timeout
}

resp, err := client.Get("https://example.com")
if err != nil {
```

```

    // If timeout exceeded: 'context deadline exceeded (Client.Timeout
    exceeded)'
    fmt.Println(err)
    return
}
defer resp.Body.Close()

body, _ := io.ReadAll(resp.Body)
fmt.Println(resp.StatusCode)
fmt.Println(string(body))

```

4.3 Full Control: `http.NewRequest()` and `client.Do()`

`http.NewRequest()` gives you complete control over the request: HTTP method, URL, headers, body, and context. `client.Do()` sends it and returns the response. This is the recommended approach for any real-world client code.

```

package main

import (
    "fmt"
    "net/http"
    "net/http/httputil"
    "net/url"
    "time"
)

func main() {
    // Parse and validate the URL
    rawURL := "https://www.golang.org"
    parsedURL, err := url.Parse(rawURL)
    if err != nil {
        fmt.Println("Invalid URL:", err)
        return
    }

    // Create client with timeout
    client := &http.Client{Timeout: 15 * time.Second}

    // Build the request
    req, err := http.NewRequest(http.MethodGet, parsedURL.String(), nil)
    if err != nil {
        fmt.Println("NewRequest:", err)
        return
    }

    // Optionally add headers
    req.Header.Set("User-Agent", "MyGoClient/1.0")
    req.Header.Set("Accept", "text/html")

    // Send the request
    resp, err := client.Do(req)
    if err != nil {

```



```

        fmt.Println("Do:", err)
        return
    }
    defer resp.Body.Close()

    // Inspect status code
    fmt.Println("Status:", resp.Status)

    // Dump response headers (useful for debugging)
    headers, _ := httputil.DumpResponse(resp, false)
    fmt.Print(string(headers))

    // Read and display content length
    contentType := resp.Header.Get("Content-Type")
    fmt.Println("Content-Type:", contentType)

    if resp.ContentLength == -1 {
        fmt.Println("ContentLength: unknown")
    } else {
        fmt.Println("ContentLength:", resp.ContentLength)
    }
}

```

The `http.MethodGet` constant is one of several defined HTTP method constants in the package:

```

http.MethodGet      = "GET"
http.MethodPost     = "POST"
http.MethodPut      = "PUT"
http.MethodPatch    = "PATCH"
http.MethodDelete   = "DELETE"
http.MethodHead     = "HEAD"
http.MethodOptions  = "OPTIONS"

```

4.4 Sending POST Requests with a Body

```

import (
    "bytes"
    "encoding/json"
    "net/http"
)

type Payload struct {
    Name  string `json:"name"`
    Value int    `json:"value"`
}

func postJSON(url string, payload Payload) error {
    data, err := json.Marshal(payload)
    if err != nil { return err }

    req, err := http.NewRequest(http.MethodPost, url, bytes.NewReader(data))

```

```

    if err != nil { return err }
    req.Header.Set("Content-Type", "application/json")

    client := &http.Client{Timeout: 10 * time.Second}
    resp, err := client.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK && resp.StatusCode !=
http.StatusCreated {
        return fmt.Errorf("unexpected status: %s", resp.Status)
    }
    return nil
}

```

4.5 Checking HTTP Status Codes

Always inspect the `StatusCode` field of the response. A successful network connection does not mean a successful HTTP operation — the server could return 404, 500, 403, etc.

```

resp, err := client.Do(req)
if err != nil {
    return fmt.Errorf("request failed: %w", err)
}
defer resp.Body.Close()

switch {
case resp.StatusCode == http.StatusOK:
    // success — read body
    body, _ := io.ReadAll(resp.Body)
    fmt.Println(string(body))

case resp.StatusCode == http.StatusNotFound:
    fmt.Println("Resource not found")

case resp.StatusCode == http.StatusUnauthorized:
    fmt.Println("Authentication required")

case resp.StatusCode >= 500:
    fmt.Printf("Server error: %s\n", resp.Status)

default:
    fmt.Printf("Unexpected status: %s\n", resp.Status)
}

```

4.6 Concurrent HTTP Requests with `errgroup`

The `golang.org/x/sync/errgroup` package makes it easy to send multiple HTTP requests concurrently, collecting the first error if any request fails. The pattern uses `g.Go()` to launch each request as a goroutine and `g.Wait()` to block until all complete.

```

package main

import (
    "fmt"
    "net/http"
    "golang.org/x/sync/errgroup"
)

func main() {
    urls := []string{
        "https://golang.org",
        "https://pkg.go.dev",
        "https://go.dev",
    }

    g := new(errgroup.Group)

    for _, url := range urls {
        url := url // capture loop variable (required before Go 1.22)
        g.Go(func() error {
            resp, err := http.Get(url)
            if err != nil {
                return err // returned to g.Wait()
            }
            defer resp.Body.Close()
            fmt.Printf("%s -> %s\n", url, resp.Status)
            return nil
        })
    }

    // Wait for all goroutines to complete
    if err := g.Wait(); err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("All requests completed successfully")
}

```

errgroup vs WaitGroup: `errgroup.Group` wraps `sync.WaitGroup` and additionally collects the first non-nil error returned by any goroutine. Use `errgroup` when any failing request should abort the operation. Use `WaitGroup` when you want to collect all results regardless of individual failures.

4.7 Command-Line HTTP Client (Statistics Service Client)

The following pattern shows how to build a real HTTP client that reads command-line flags and sends requests to a running HTTP service — the cobra-based CLI client for the statistics web service:

```

// Reading global flags (server and port) from cobra/viper:
SERVER := viper.GetString("server") // default: "localhost"
PORT   := viper.GetString("port")   // default: "1234"

// — status command —
URL := "http://" + SERVER + ":" + PORT + "/status"
resp, err := http.Get(URL)
if err != nil { fmt.Println(err); return }
defer resp.Body.Close()
if resp.StatusCode != http.StatusOK {
    fmt.Println("Status code:", resp.StatusCode)
    return
}
body, _ := io.ReadAll(resp.Body)
fmt.Print(string(body))

// — search command —
dataset, _ := cmd.Flags().GetString("dataset")
URL = "http://" + SERVER + ":" + PORT + "/search/" + dataset
// ... same pattern: Get -> check status -> read body

// — insert command (building a value URL) —
values, _ := cmd.Flags().GetString("values") // e.g. "1,2,3,-4,0"
VALS := strings.Split(values, ",")
vSend := ""
for _, v := range VALS {
    if _, err := strconv.ParseFloat(v, 64); err == nil {
        vSend += "/" + v
    }
}
URL = "http://" + SERVER + ":" + PORT + "/insert/" + dataset + vSend + "/"
resp, err = http.Get(URL)
// ... check status -> read body

```

5. Timing Out HTTP Connections

Timeouts are critical in any production HTTP application. Without them, slow or unresponsive peers can exhaust goroutines, file descriptors, and memory. Go provides multiple layers at which you can configure timeouts.

5.1 Timeout Architecture

Timeout Type	Set On	What It Controls
Client.Timeout	http.Client	The entire request from start to finish: dial + TLS + send request + read response headers + read response body.
Transport.TLSHandshakeTimeout	http.Transport	Time allowed for the TLS handshake when connecting to HTTPS endpoints.
Transport.ResponseHeaderTimeout	http.Transport	Time to wait for server response headers after sending the request.
Transport.IdleConnTimeout	http.Transport	How long an idle keep-alive connection stays in the pool before closing.
Server.ReadTimeout	http.Server	Max time to read the entire incoming request, including body. Protects from slow senders.
Server.ReadHeaderTimeout	http.Server	Max time to read only the request headers. More targeted than ReadTimeout.
Server.WriteTimeout	http.Server	Max time to write the response to the client. Protects from slow receivers.
Server.IdleTimeout	http.Server	Max time to wait for the next request on a keep-alive connection.
context.WithTimeout / WithDeadline	Request Context	Per-request timeout that cancels the request after the deadline, usable with any client.

5.2 Client Timeout: http.Client.Timeout

The simplest and most complete way to timeout a client request is to set Timeout on http.Client. This timeout covers the entire lifecycle — DNS lookup, TCP connect, TLS handshake, request send, response header read, and response body read.

```

client := &http.Client{
    Timeout: 10 * time.Second, // 10 seconds for the entire request
}

resp, err := client.Get("http://slow-server.example.com/data")
if err != nil {
    // err contains: 'context deadline exceeded (Client.Timeout exceeded)'
    fmt.Println("Request timed out:", err)
    return
}
defer resp.Body.Close()

```

5.3 Per-Request Timeout with context.WithTimeout

For per-request timeouts (different timeout per request from the same client), use `context.WithTimeout()` combined with `http.NewRequestWithContext()`. The context is cancelled after the given duration, which immediately cancels the request.

```

package main

import (
    "context"
    "fmt"
    "io"
    "net/http"
    "time"
)

func fetchWithTimeout(url string, timeout time.Duration) (string, error) {
    // Create a context that cancels after 'timeout'
    ctx, cancel := context.WithTimeout(context.Background(), timeout)
    defer cancel() // always call cancel to release resources

    // Build request with the context attached
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return "", err
    }

    // Send - if timeout elapses, Do() returns immediately with context error
    resp, err := http.DefaultClient.Do(req)
    if err != nil {
        return "", fmt.Errorf("request failed: %w", err)
    }
    defer resp.Body.Close()

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", err
    }
    return string(body), nil
}

```

```

func main() {
    result, err := fetchWithTimeout("http://localhost:1234/status",
    5*time.Second)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println(result)
}

```

```

# Example output when timeout is exceeded:
$ go run timeoutClient.go http://localhost:1234 5
Delay: 5
Get "http://localhost:1234": context deadline exceeded

```

5.4 Transport-Level Timeout with `net.DialTimeout` and `SetDeadline`

For more granular control, you can provide a custom Dial function to `http.Transport` that uses `net.DialTimeout` and `conn.SetDeadline`. This lets you separate the connection timeout from the read/write deadline.

```

var timeout = 1 * time.Second

// Custom dialer with deadline
func dialWithDeadline(network, host string) (net.Conn, error) {
    // DialTimeout: max time to establish the TCP connection
    conn, err := net.DialTimeout(network, host, timeout)
    if err != nil {
        return nil, err
    }
    // SetDeadline: absolute deadline for all subsequent I/O on the
    connection
    // Any Read or Write that does not complete before this time returns an
    error
    conn.SetDeadline(time.Now().Add(timeout))
    return conn, nil
}

func main() {
    // Inject the custom dialer into Transport
    t := &http.Transport{
        Dial: dialWithDeadline,
    }
    client := &http.Client{Transport: t}

    resp, err := client.Get("http://example.com")
    if err != nil {
        fmt.Println("Error:", err) // 'i/o timeout' if deadline exceeded
        return
    }
    defer resp.Body.Close()
}

```

```
    io.Copy(os.Stdout, resp.Body)
}
```

```
# Successful request (completed within 1 second):
$ go run withDeadline.go http://www.golang.org
Timeout value: 1s
<!DOCTYPE html>...

# Failed request (server did not respond in time):
$ go run withDeadline.go http://localhost:80
Timeout value: 1s
Get "http://localhost:80": read tcp 127.0.0.1:52492->127.0.0.1:80: i/o
timeout
```

5.5 Server-Side Timeouts

Server timeouts are equally important. Without them, malicious or slow clients can hold connections open indefinitely (Slowloris attack) and prevent the server from accepting new connections.

```
srv := &http.Server{
    Addr:    ":8001",
    Handler: mux,

    // ReadTimeout: max time to read entire request including body
    // A slow-uploading client gets cut off after this duration
    ReadTimeout: 3 * time.Second,

    // ReadHeaderTimeout: max time to receive the request headers only
    // More specific than ReadTimeout; recommended to set both
    ReadHeaderTimeout: 1 * time.Second,

    // WriteTimeout: max time from end-of-request-headers to end-of-response
    // If the handler takes longer than this, the connection is closed
    WriteTimeout: 3 * time.Second,

    // IdleTimeout: max time to wait for next request on a keep-alive
    connection
    // Lower values free connections faster; higher values reduce reconnect
    overhead
    IdleTimeout: 60 * time.Second,
}

if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
    log.Fatal(err)
}
```

Timeout Field	Typical Value	When to Adjust
---------------	---------------	----------------

ReadTimeout	5–30s	Increase for APIs that accept large file uploads.
ReadHeaderTimeout	2–5s	Usually smaller than ReadTimeout; protects from header-flooding attacks.
WriteTimeout	5–30s	Increase for endpoints that generate large responses or stream data.
IdleTimeout	30–120s	Increase for high-frequency clients to reduce reconnect overhead.

5.6 Choosing the Right Timeout Strategy

Scenario	Recommended Approach
Simple script or one-off tool	<code>http.Client{Timeout: 30*time.Second}</code> is sufficient.
Production microservice client	Custom <code>http.Client</code> + <code>http.Transport</code> with per-host connection limits, TLS timeout, and response header timeout.
Per-request variable timeout	<code>context.WithTimeout()</code> + <code>http.NewRequestWithContext()</code> for fine-grained per-call control.
Production HTTP server	Set <code>ReadTimeout</code> , <code>ReadHeaderTimeout</code> , <code>WriteTimeout</code> , and <code>IdleTimeout</code> on <code>http.Server</code> .
Streaming responses (SSE, large files)	Set <code>WriteTimeout = 0</code> (disabled) or use per-stream context cancellation; a fixed <code>WriteTimeout</code> would cut off legitimate slow consumers.

6. Advanced Topics

6.1 HTTPS with TLS

```
// Server: serve HTTPS with a cert and key file
srv := &http.Server{
    Addr:    ":443",
    Handler: mux,
}
err := srv.ListenAndServeTLS("cert.pem", "key.pem")

// Client: skip TLS certificate verification (INSECURE - testing only)
client := &http.Client{
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    },
}

// Client: use a custom CA certificate
certPool, _ := x509.SystemCertPool()
caCert, _ := os.ReadFile("ca.pem")
certPool.AppendCertsFromPEM(caCert)
client = &http.Client{
    Transport: &http.Transport{
        TLSClientConfig: &tls.Config{RootCAs: certPool},
    },
}
```

6.2 Graceful Server Shutdown

`http.Server.Shutdown()` stops the server gracefully: it stops accepting new connections and waits for active requests to complete before returning. This is essential for zero-downtime deployments.

```
srv := &http.Server{Addr: ":8080", Handler: mux}

// Start server in background
go func() {
    if err := srv.ListenAndServe(); err != http.ErrServerClosed {
        log.Fatalf("Server error: %v", err)
    }
}()

// Listen for OS signals (Ctrl+C, SIGTERM)
quit := make(chan os.Signal, 1)
signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
<-quit

// Give active requests 30 seconds to complete
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
```

```

if err := srv.Shutdown(ctx); err != nil {
    log.Printf("Shutdown error: %v", err)
} else {
    log.Println("Server stopped gracefully")
}

```

6.3 Middleware Pattern

Middleware wraps an `http.Handler` to add cross-cutting concerns (logging, authentication, CORS, rate limiting) without modifying the handler itself. A middleware function takes an `http.Handler` and returns an `http.Handler`.

```

// Logging middleware
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        log.Printf("--> %s %s", r.Method, r.URL.Path)
        next.ServeHTTP(w, r) // call the wrapped handler
        log.Printf("<-- %s %s (%v)", r.Method, r.URL.Path, time.Since(start))
    })
}

// Auth middleware
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if token != "Bearer secret" {
            http.Error(w, "Unauthorized", http.StatusUnauthorized)
            return
        }
        next.ServeHTTP(w, r)
    })
}

// Chaining middleware
func chain(h http.Handler, middlewares ...func(http.Handler) http.Handler) http.Handler {
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/api/", apiHandler)

    // Apply middleware: requests pass through logging -> auth -> handler
    handler := chain(mux, loggingMiddleware, authMiddleware)
    http.ListenAndServe(":8080", handler)
}

```

6.4 Reading Response Headers with `httputil.DumpResponse`

`httputil.DumpResponse` serializes an HTTP response for debugging. The second argument controls whether the body is included. `httputil.DumpRequest` does the same for incoming requests on the server side.

```
import "net/http/httputil"

resp, _ := client.Do(req)
defer resp.Body.Close()

// Dump headers only (body=false)
headerDump, _ := httputil.DumpResponse(resp, false)
fmt.Print(string(headerDump))

// Dump headers + body (body=true)
// WARNING: this reads the entire body - you won't be able to read it again
fullDump, _ := httputil.DumpResponse(resp, true)
fmt.Print(string(fullDump))

// On the server side, dump the incoming request:
func debugHandler(w http.ResponseWriter, r *http.Request) {
    dump, _ := httputil.DumpRequest(r, true)
    fmt.Printf("%s\n", dump)
}
```

6.5 Docker Deployment of HTTP Services

Go HTTP services are commonly packaged as minimal Docker images using multi-stage builds. The builder stage compiles the binary; the final stage contains only the binary and a minimal OS image.

```
# Dockerfile (multi-stage build)
FROM golang:alpine AS builder
RUN apk add --no-cache git
RUN mkdir $GOPATH/src/server
ADD ./stats.go $GOPATH/src/server
ADD ./handlers.go $GOPATH/src/server
WORKDIR $GOPATH/src/server
RUN go mod init && go mod tidy
RUN go build -o /app/server stats.go handlers.go

FROM alpine:latest
COPY --from=builder /app/server /app/server
EXPOSE 1234
WORKDIR /app
CMD ["/app/server"]
```

```
# docker-compose.yml
version: "3"
services:
  goapp:
    image: goapp
    container_name: goapp
    restart: always
    ports:
      - "1234:1234"
    networks:
      - services
networks:
  services:
    driver: bridge
```

```
# Build and run:
$ docker build -f Dockerfile -t goapp .
$ docker-compose up

# Test:
$ curl localhost:1234/status
Total entries: 5
```

7. Quick Reference Cheat Sheet

Starting an HTTP Server

```
// Minimal (uses DefaultServeMux)
http.HandleFunc("/", handler)
http.ListenAndServe(":8080", nil)

// Production (custom ServeMux + Server)
mux := http.NewServeMux()
mux.HandleFunc("/path", handler)
srv := &http.Server{
    Addr: ":8080", Handler: mux,
    ReadTimeout: 5*time.Second, WriteTimeout: 10*time.Second,
    IdleTimeout: 120*time.Second,
}
srv.ListenAndServe()

// HTTPS
srv.ListenAndServeTLS("cert.pem", "key.pem")

// Graceful shutdown
srv.Shutdown(ctx)
```

Handler Functions

```
// Signature
func handler(w http.ResponseWriter, r *http.Request) { }

// Write response
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated) // must be before Write()
fmt.Fprintf(w, `{"key":"val"}`)
http.Error(w, "Not found", http.StatusNotFound) // shortcut

// Read request
r.Method      // "GET", "POST", ...
r.URL.Path     // "/api/users"
r.URL.Query().Get("key") // query string
r.Header.Get("Authorization")
io.ReadAll(r.Body); r.Body.Close()
r.RemoteAddr   // "1.2.3.4:5678"
r.Context()    // request context
```

HTTP Clients

```
// Simple GET
resp, err := http.Get(url)
defer resp.Body.Close()
```

```

body, _ := io.ReadAll(resp.Body)

// Custom client with timeout (recommended for production)
client := &http.Client{Timeout: 15 * time.Second}
resp, err := client.Get(url)

// Full control
req, _ := http.NewRequest(http.MethodPost, url, bodyReader)
req.Header.Set("Content-Type", "application/json")
resp, err := client.Do(req)

// With per-request timeout
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
resp, err := client.Do(req)

// Check status
if resp.StatusCode != http.StatusOK { /* handle error */ }

```

Timeout Summary

```

// Client - entire request lifecycle
client := &http.Client{Timeout: 30 * time.Second}

// Client - granular transport timeouts
transport := &http.Transport{
    TLSHandshakeTimeout: 10 * time.Second,
    ResponseHeaderTimeout: 10 * time.Second,
    IdleConnTimeout: 90 * time.Second,
}

// Per-request timeout
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)

// Server timeouts
srv := &http.Server{
    ReadTimeout: 5 * time.Second,
    ReadHeaderTimeout: 2 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout: 120 * time.Second,
}

```

Common Status Code Constants

http.StatusOK	= 200
http.StatusCreated	= 201
http.StatusNoContent	= 204
http.StatusMovedPermanently	= 301

```
http.StatusNotModified      = 304
http.StatusBadRequest      = 400
http.StatusUnauthorized     = 401
http.StatusForbidden        = 403
http.StatusNotFound         = 404
http.StatusMethodNotAllowed = 405
http.StatusInternalServerError = 500
http.StatusBadGateway       = 502
http.StatusServiceUnavailable = 503
```

Golden Rule: Always close `resp.Body`, always set server timeouts, always use a custom `http.Client` with `Timeout` in production, and always check the HTTP status code — not just the Go error. A nil error means the request was delivered; the status code tells you whether it succeeded.